

System 13: High-Throughput Logging Framework

Zero-Prerequisite Intuition: The "Airport Black Box" Metaphor What is a logging framework, and why can't we just use `print()`? Imagine an airplane in mid-flight. Hundreds of sensors (engine temperature, altitude, hydraulic pressure) record telemetry every millisecond. If the pilot had to stop flying the airplane and personally hand-write every temperature reading on paper before pulling the steering wheel, the plane would crash!

Instead, sensors dump readings into an automated **Black Box** in the background while the pilot continues flying uninterrupted.

In high-scale web servers, every request generates logs. If an application thread blocks on a synchronous disk write (`file.write()`) on every request, your server throughput drops from 10,000 requests/sec to 100 requests/sec because hard drives and networks are 100,000x slower than CPU RAM!

The **#1 Low-Level Design question at Uber, Stripe, and Google** is: *"Design a multi-threaded, asynchronous, high-throughput logging library that supports log levels, custom formatters, multiple destinations (Console, File, S3), and never blocks worker threads."*

1. Requirements & System Boundaries

Functional Requirements

1. **Log Levels:** Strict priority filtering: `DEBUG (10) < INFO (20) < WARN (30) < ERROR (40) < FATAL (50)`.
2. **Pluggable Appenders / Sinks:** Support writing to multiple destinations simultaneously (Console, File, Remote Network).
3. **Pluggable Formatters:** Support converting log records into Human-Readable Plain Text or Machine-Readable JSON.

Non-Functional Requirements

- **Zero Latency Overhead (Asynchronous):** Worker threads log in $< 1 \mu s$ by pushing into an in-memory queue. A background thread handles batch flushing to disk.
- **Buffer Overflow Policy:** What happens when logs arrive faster than disk can write? (Drop lowest priority logs vs. block producer).

2. Design Patterns Architecture

```
classDiagram
    direction TB
    class LogLevel {
        <<enumeration>>
        DEBUG
        INFO
        WARN
        ERROR
        FATAL
    }
    class LogMessage {
        +LogLevel level
        +str message
        +str logger_name
        +float timestamp
        +int thread_id
    }
    class LogFormatter {
        <<interface>>
        +format(message)* str
    }
    class JSONFormatter {
        +format(message) str
    }
    class TextFormatter {
        +format(message) str
    }
    class LogSink {
        <<interface>>
        +write(formatted_text)* void
        +flush()* void
    }
    class ConsoleSink {
        +write(formatted_text) void
    }
    class FileSink {
        +write(formatted_text) void
    }
    class AsyncLogger {
```

```
-Queue buffer
-Thread worker
-LogLevel threshold
-list[LogSink] sinks
+log(level, msg) void
+shutdown() void
}

LogFormatter <| .. JSONFormatter
LogFormatter <| .. TextFormatter
LogSink <| .. ConsoleSink
LogSink <| .. FileSink
AsyncLogger o→ LogSink
AsyncLogger o→ LogFormatter
AsyncLogger ..> LogMessage
```

3. Complete Python Implementation

```
# async_logger.py
import queue
import threading
import time
import json
from abc import ABC, abstractmethod
from enum import IntEnum
from dataclasses import dataclass
from typing import List, Optional

# =====
# 1. Log Levels
# =====

class LogLevel(IntEnum):
    DEBUG = 10
    INFO = 20
    WARN = 30
    ERROR = 40
    FATAL = 50

@dataclass(frozen=True)
class LogRecord:
    level: LogLevel
    message: str
    logger_name: str
    timestamp: float
    thread_id: int

# =====
# 2. Strategy Pattern: Formatters
# =====

class LogFormatter(ABC):
    @abstractmethod
    def format(self, record: LogRecord) → str:
        pass

class TextFormatter(LogFormatter):
    def format(self, record: LogRecord) → str:
        t_str = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(record.timestamp))
```

```

        return f"[{t_str}] [{record.level.name:5}] [{record.logger_name}] (Thread-{record.thread_id}) {record.message}"

class JSONFormatter(LogFormatter):
    def format(self, record: LogRecord) → str:
        payload = {
            "timestamp": record.timestamp,
            "level": record.level.name,
            "logger": record.logger_name,
            "thread_id": record.thread_id,
            "message": record.message
        }
        return json.dumps(payload)

# =====
# 3. Strategy Pattern: Sinks (Appenders)
# =====

class LogSink(ABC):
    def __init__(self, formatter: LogFormatter):
        self.formatter = formatter

    @abstractmethod
    def emit(self, record: LogRecord) → None:
        pass

    @abstractmethod
    def flush(self) → None:
        pass

class ConsoleSink(LogSink):
    def emit(self, record: LogRecord) → None:
        formatted = self.formatter.format(record)
        print(formatted)

    def flush(self) → None:
        pass

class MemoryBufferSink(LogSink):
    """Sink used for unit testing or high-speed in-memory analytics."""
    def __init__(self, formatter: LogFormatter):
        super().__init__(formatter)
        self.entries: List[str] = []

```

```

def emit(self, record: LogRecord) → None:
    self.entries.append(self.formatter.format(record))

def flush(self) → None:
    pass

# =====
# 4. Asynchronous Producer-Consumer Engine
# =====

class AsyncLogger:
    def __init__(self, name: str, threshold: LogLevel = LogLevel.INFO, max_buffer_size: int = 1000):
        self.name = name
        self.threshold = threshold
        self.sinks: List[LogSink] = []
        self._queue = queue.Queue(maxsize=max_buffer_size)
        self._shutdown_event = threading.Event()

        # Background consumer thread
        self._worker_thread = threading.Thread(target=self._drain_queue, daemon=True, name=f'AsyncLogger-Worker-{self.name}')
        self._worker_thread.start()

    def add_sink(self, sink: LogSink) → None:
        self.sinks.append(sink)

    def log(self, level: LogLevel, message: str) → None:
        # Fast filter: Drop messages below log threshold immediately!
        if level < self.threshold:
            return

        record = LogRecord(
            level=level,
            message=message,
            logger_name=self.name,
            timestamp=time.time(),
            thread_id=threading.get_ident()
        )

        try:
            # Non-blocking enqueue: takes < 1 microsecond!
            self._queue.put_nowait(record)
        except queue.Full:
            # Drop policy for overload protection

```

```
print(f"[LOGGER OVERLOAD] Queue full! Dropping log from {self.name}")

def info(self, msg: str): self.log(LogLevel.INFO, msg)
def error(self, msg: str): self.log(LogLevel.ERROR, msg)
def warn(self, msg: str): self.log(LogLevel.WARN, msg)
def debug(self, msg: str): self.log(LogLevel.DEBUG, msg)

def _drain_queue(self):
    """Dedicated background thread flushing log messages to all sinks."""
    while not self._shutdown_event.is_set() or not self._queue.empty():
        try:
            record = self._queue.get(timeout=0.1)
            for sink in self.sinks:
                try:
                    sink.emit(record)
                except Exception as e:
                    print(f"Sink emission error: {e}")
            self._queue.task_done()
        except queue.Empty:
            continue

def shutdown(self):
    """Gracefully drain the remaining queue before exiting."""
    self._shutdown_event.set()
    self._worker_thread.join(timeout=2.0)
    for sink in self.sinks:
        sink.flush()
```

4. Verification & Stress-Testing

```
if __name__ == "__main__":
    logger = AsyncLogger("PaymentService", threshold=LogLevel.DEBUG)

    # Attach two independent sinks: Plain text Console + JSON Memory
    logger.add_sink(ConsoleSink(TextFormatter()))
    mem_sink = MemoryBufferSink(JSONFormatter())
    logger.add_sink(mem_sink)

    # Multi-threaded stress test: 5 threads logging concurrently
    def simulate_traffic(user_id):
        logger.info(f"User {user_id} requested checkout")
        logger.debug(f"User {user_id} token verified")
        if user_id % 3 == 0:
            logger.error(f"User {user_id} payment declined")

    threads = [threading.Thread(target=simulate_traffic, args=(i,)) for i in range(10)]
    for t in threads: t.start()
    for t in threads: t.join()

    # Graceful flush
    logger.shutdown()

    print(f"\nTotal JSON logs in memory buffer: {len(mem_sink.entries)}")
    print("Sample JSON entry:\n" + mem_sink.entries[0])
```